L10 — Spring Boot REST API + JPA/ORM

10.1 Steps to set up Spring Boot + REST + JPA (Maven deps: spring-boot-starter-web, spring-boot-starter-data-jpa, MySQL connector) and role of embedded Tomcat.

Answer:

Spring Boot makes it easier to create Java web applications and REST APIs.

Required Maven dependencies:

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
   <groupId>com.mysql</groupId>
   <artifactId>mysql-connector-j</artifactId>
   <scope>runtime</scope>
</dependency>
```

Steps:

1. Create Spring Boot project.
2. Add Web dependency.
3. Add Spring Data JPA.
4. Add MySQL connector.
5. Configure database.
6. Create Entity.
7. Create Repository.
8. Create REST Controller.
9. Run application.
10. Test using browser/Postman.

Embedded Tomcat

Spring Boot normally includes an **embedded Tomcat server** when using the web starter.

Therefore, we generally do not need to install and configure a separate Tomcat server to run the application

10.2 JPA Student entity (id, name, cgpa) + StudentRepository extends JpaRepository.**Entity:**

```
import jakarta.persistence.*;

@Entity
@Table(name = "Students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private double cgpa;

    public Student() {
    }

    public Student(String name, double cgpa) {
        this.name = name;
        this.cgpa = cgpa;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getCgpa() {
        return cgpa;
    }

    public void setCgpa(double cgpa) {
        this.cgpa = cgpa;
    }
}
```

Repository:

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository
        extends JpaRepository<Student, Long> {
}
```

Explanation:

@Entity tells JPA that Student is a database entity.

@Id identifies the primary key.

@GeneratedValue automatically generates ID values.

JpaRepository provides built-in methods such as:

save()
findAll()
findById()
deleteById()

So we don't need to manually write SQL for basic CRUD operations.

10.3 @RestController with GET /students and POST /students endpoints using the repository.

Answer:

```
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

    private final StudentRepository repository;

    public StudentController(
        StudentRepository repository) {

        this.repository = repository;
    }

    @GetMapping
    public List<Student> getStudents() {
```

```
        return repository.findAll();
    }

    @PostMapping
    public Student addStudent(
        @RequestBody Student student) {

        return repository.save(student);
    }
}
```

GET:

Request:

GET /students

It returns all students.

POST:

Request:

POST /students

JSON:

```
{
    "name": "Rahim",
    "cgpa": 3.75
}
```

The repository saves the student into the database.

Flow:

Client
↓
REST Controller
↓
Repository
↓
JPA / Hibernate
↓
MySQL

L11 — Servlet CRUD — District Quiz Game

11.1 Design DB schema for quiz on Crops / Geography / Academic Institutions of your district + Player/Score table. Justify design.

Answer:

For a district-based quiz game, we can create separate tables for players, questions, and scores.

Player Table

```
CREATE TABLE Player (
   player_id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100) NOT NULL
);
```

Question Table

```
CREATE TABLE Question (
   question_id INT PRIMARY KEY AUTO_INCREMENT,
   category VARCHAR(50),
   question_text VARCHAR(255),
   option_a VARCHAR(100),
   option_b VARCHAR(100),
   option_c VARCHAR(100),
   option_d VARCHAR(100),
   correct_answer CHAR(1)
);
```

The category can contain:

Crops
Geography
Academic Institution

PlayerScore Table

```
CREATE TABLE PlayerScore (
   score_id INT PRIMARY KEY AUTO_INCREMENT,
   player_id INT,
   score INT,
   FOREIGN KEY (player_id)
      REFERENCES Player(player_id)
);
```

Example questions:

Category: Crops
Question: Which crop is commonly grown in the district?

Category: Geography
Question: Which river is associated with the district?

Category: Academic Institution
Question: Which institution is located in the district?

Why this design?

The database is separated into different tables to avoid unnecessary duplication.

The relationship is:

Player
  |
  | 1
  |
  | many
PlayerScore

The question table stores reusable quiz questions.

11.2 Servlet method: save Player name + final score into PlayerScore via JDBC/PreparedStatement.

Answer:

```
protected void saveScore(
   HttpServletRequest request,
   HttpServletResponse response)
   throws IOException {

   String name =
      request.getParameter("name");

   int score = Integer.parseInt(
      request.getParameter("score")
   );

   String url =
```

```
    "jdbc:mysql://localhost:3306/quiz_db";

String username = "root";
String password = "1234";

try {

    Connection con =
        DriverManager.getConnection(
            url, username, password
        );

    // Insert player
    String playerSQL =
        "INSERT INTO Player(name) VALUES (?)";

    PreparedStatement ps1 =
        con.prepareStatement(
            playerSQL,
            Statement.RETURN_GENERATED_KEYS
        );

    ps1.setString(1, name);
    ps1.executeUpdate();

    ResultSet keys =
        ps1.getGeneratedKeys();

    int playerId = 0;

    if (keys.next()) {
        playerId = keys.getInt(1);
    }

    // Insert score
    String scoreSQL =
        "INSERT INTO PlayerScore(player_id, score) " +
        "VALUES (?, ?)";

    PreparedStatement ps2 =
        con.prepareStatement(scoreSQL);

    ps2.setInt(1, playerId);
    ps2.setInt(2, score);

    ps2.executeUpdate();
```

```
            response.getWriter().println(
                "Score saved successfully."
            );

            ps1.close();
            ps2.close();
            con.close();

        } catch (SQLException e) {

            response.getWriter().println(
                "Database Error: " + e.getMessage()
            );
        }
}
```

Important:

PreparedStatement is used to safely insert values into the database.

11.3 Quiz logic: ≥3 MCQs (crop, geography, institution) as objects; present, check answer, increment score.

Answer:

We can represent each question using a Java object.

```
class Question {

    String question;
    String[] options;
    int correctAnswer;

    Question(
        String question,
        String[] options,
        int correctAnswer) {

        this.question = question;
        this.options = options;
        this.correctAnswer = correctAnswer;
    }
}
```

Create Questions:

```
Question q1 = new Question(
   "Which crop is common in the district?",
   new String[]{"Rice", "Tea", "Wheat", "Coffee"},
   0
);

Question q2 = new Question(
   "Which river is important in the district?",
   new String[]{"Padma", "Surma", "Karnaphuli", "Teesta"},
   1
);

Question q3 = new Question(
   "Which is an academic institution?",
   new String[]{
      "District College",
      "Airport",
      "Market",
      "Bridge"
   },
   0
);
```

Check Answers:

```
int score = 0;

int userAnswer = 0;

if (userAnswer == q1.correctAnswer) {
   score++;
}
```

For multiple questions:

```
Question[] questions = {q1, q2, q3};

int score = 0;

for (Question q : questions) {

   // userAnswer comes from the form

   if (userAnswer == q.correctAnswer) {
```

```
        score++;
    }
}

System.out.println("Final Score: " + score);
```

Logic:

```
Show Question
    ↓
Show 4 Options
    ↓
User selects answer
    ↓
Compare with correct answer
    ↓
Correct? → score++
    ↓
Next question
    ↓
Final Score
    ↓
Save to Database
```

L12 — GoF Design Patterns

12.1 List all 5 GoF Creational patterns (Singleton, Factory Method, Abstract Factory, Builder, Prototype) with one-line purpose each.The **Gang of Four (GoF)** defines five creational design patterns.

Pattern	Purpose
Singleton	Ensures a class has only one object/instance and provides a global access point
Factory Method	Creates objects without requiring the client to know the exact concrete class
Abstract Factory	Creates families of related objects
Builder	Builds complex objects step-by-step
Prototype	Creates new objects by copying/cloning an existing object

Easy memory:

Singleton → One

Factory → Create

Abstract Factory → Family

Builder → Step-by-step

Prototype → Copy

12.2 List all 7 GoF Structural patterns (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) with one-line purpose each.The seven GoF structural patterns are:

Pattern	Purpose
Adapter	Makes incompatible interfaces work together
Bridge	Separates abstraction from implementation
Composite	Treats individual objects and groups of objects uniformly
Decorator	Adds behavior to an object dynamically
Facade	Provides a simple interface to a complex system
Flyweight	Shares objects to reduce memory usage
Proxy	Provides a substitute or controlled access to another object

Easy memory:

- **Adapter → Connect incompatible things**
- **Bridge → Separate abstraction/implementation**
- **Composite → Tree/group structure**
- **Decorator → Add behavior**
- **Facade → Simple front door**
- **Flyweight → Share objects**
- **Proxy → Substitute/control access**

12.3 Implement Singleton (thread-safe) and Adapter with short, complete Java code examples.**A. Singleton Pattern**

Answer:

Singleton ensures that a class has **only one instance**.

A simple thread-safe approach is to use a private constructor and a synchronized access method.

```
class Singleton {
```

```
    private static Singleton instance;

    private Singleton() {
    }

    public static synchronized Singleton getInstance() {

        if (instance == null) {
            instance = new Singleton();
        }

        return instance;
    }

    public void showMessage() {
        System.out.println(
            "Singleton object is working."
        );
    }
}

public class Main {

    public static void main(String[] args) {

        Singleton s1 =
            Singleton.getInstance();

        Singleton s2 =
            Singleton.getInstance();

        s1.showMessage();

        System.out.println(
            s1 == s2
        );
    }
}
```

Output:

Singleton object is working.
true

Because:

s1 == s2

is true, both references point to the **same object**.

Why thread-safe?

The synchronized keyword ensures that multiple threads cannot simultaneously execute getInstance() in a way that creates multiple instances.

B. Adapter Pattern

Answer:

Adapter is used when **two incompatible interfaces need to work together**.

Example:

Suppose our application expects:

Target

but an existing class provides:

Adaptee

The Adapter connects them.

```
interface Printer {
   void print();
}

class OldPrinter {

   public void oldPrint() {
      System.out.println(
         "Printing using old printer."
      );
   }
}

class PrinterAdapter implements Printer {

   private OldPrinter oldPrinter;
```

```
    public PrinterAdapter(
        OldPrinter oldPrinter) {

        this.oldPrinter = oldPrinter;
    }

    @Override
    public void print() {
        oldPrinter.oldPrint();
    }
}

public class Main {

    public static void main(String[] args) {

        OldPrinter oldPrinter =
            new OldPrinter();

        Printer printer =
            new PrinterAdapter(oldPrinter);

        printer.print();
    }
}
```

Output:

Printing using old printer.

Structure:

Client
↓
Printer interface
↓
PrinterAdapter
↓
OldPrinter

The Adapter allows the new Printer interface to work with the existing OldPrinter class.